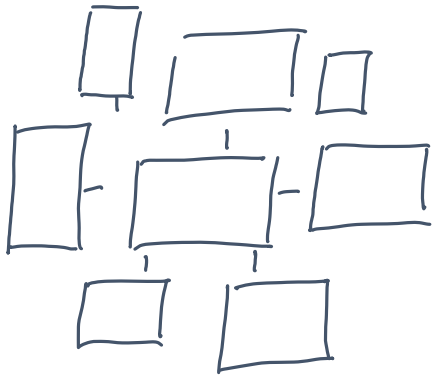


Embedded Signal Processing Systems

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

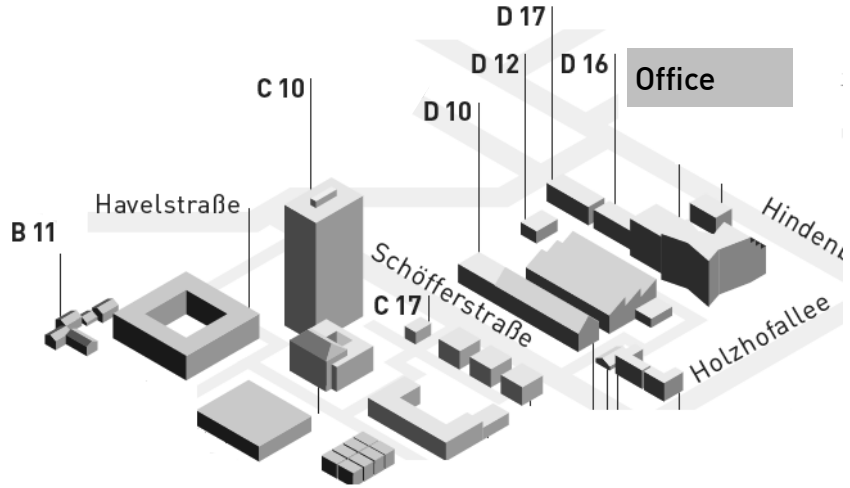
University of Applied Sciences Darmstadt
h_da
Faculty of Electrical Engineering and Information Technology
fbeit

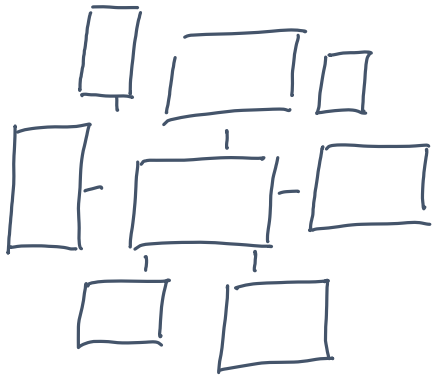
Personal Data

- Name: Prof. Dr. Christian Jakob
- Office: 311/D16, Birkenweg 8, 64295 Darmstadt
- Email: christian.jakob@h-da.de
- Phone: +49.6151.16-37706

- Office hour: (most probably) Monday, 9am
  in room 311/D16

  by appointment only





Embedded Signal Processing Systems

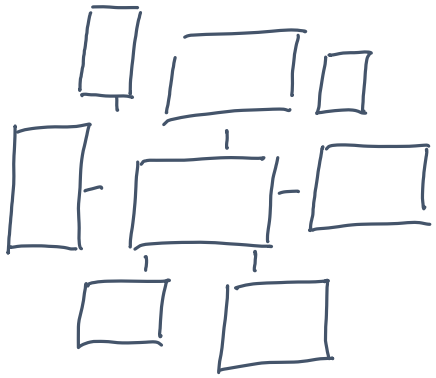
International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob
University of Applied Sciences Darmstadt
h_da
Faculty of Electrical Engineering and Information Technology
fbeit

Introduction to SystemVerilog

Today's Agenda

- Roadmap for the next two days ...
- A simple DDFS architecture - An introductory project ...
- Logic simulation using Mentor Modelsim
- Lab@Home Exercises



Roadmap for the next two lecture sessions ...

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt
h_da
Faculty of Electrical Engineering and Information Technology
fbeit

Roadmap for the next two days …

- **Today**: A short glimpse on what we intend to study this term …

Roadmap for the next two days ...

- **Today**: FPGA based DSP Design in a Nutshell ... An introductory project

Module content, Course Goals and Intended Learning Outcome

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob
University of Applied Sciences Darmstadt
h_da
Faculty of Electrical Engineering and Information Technology
fbeit

Module Content

- The aim of this course is to provide students with a solid understanding of designing complex embedded signal processing systems using modern **µC** and **FPGA** architectures.

Module Content

- The aim of this course is to provide students with a solid understanding of designing complex embedded signal processing systems using modern µC and FPGA architectures.
- Key subjects are the **design**, **modelling** and **simulation of fixed-point DSP algorithms** as well as their **HW/SW implementation** on state-of-the-art processing platforms. In particular, the course will cover:

Module Content

- The aim of this course is to provide students with a solid understanding of designing complex embedded signal processing systems using modern µC and FPGA architectures.
- Key subjects are the design, modelling and simulation of fixed-point DSP algorithms as well as their HW/SW implementation on state-of-the-art processing platforms. In particular, the course will cover:

  - an introduction to modern DSP systems – Emerging applications, architectures and challenges.
  - the theory of discrete-time systems and fixed-point mathematics.
  - the design and implementation of digital filters (FIR/IIR digital filter design and specification, re-timing: cut-set and delay scaling, the transpose FIR, pipelining and multichannel architectures).
  - the synthesis of digital signals (NCO Design, DDFS, CORDIC algorithm, IIR oscillators).

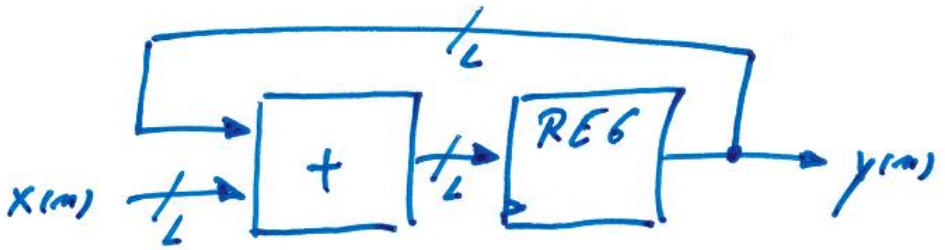
Module Content

- The aim of this course is to provide students with a solid understanding of designing complex embedded signal processing systems using modern µC and FPGA architectures.
- Key subjects are the design, modelling and simulation of fixed-point DSP algorithms as well as their HW/SW implementation on state-of-the-art processing platforms. In particular, the course will cover:

  - digital correlator architectures (Auto/cross-correlation techniques).
  - the Discrete Fourier Transform, various FFT algorithms and architectures, as well as design issues related to FFT word-length growth and accuracy.
  - HLS and Model based DSP design: Synthesis of custom DSP accelerators
  - Design and implementation of digital control systems: Mapping analog control loops to digital platforms.

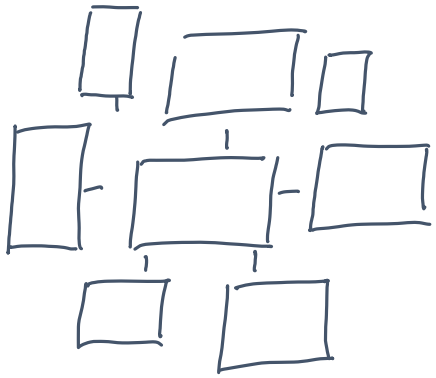
Module Content

- The lab focuses on teaching practical skills related to the design and implementation of embedded signal processing systems using C and SystemVerilog:
  - Analysis, modelling and simulation of various DSP algorithms.
  - Mapping DSP algorithms (Filters, signal synthesisers) to µC and FPGA platforms followed by profiling and benchmarking of the respective HW/SW solutions.



Course Goals and Intended Learning Outcome

- to understand:
  - the architectural features of modern DSP processing systems.
  - the tools and methodologies for embedded DSP design.
  - the basic strategies for mapping algorithms to HW and SW platforms.
- to apply:
  - the gained knowledge to analyse, model and simulate dedicated DSP algorithms.
  - the gained knowledge to map a given floating-point DSP algorithm to its fixed-point equivalent.
  - the gained knowledge to implement fixed-point algorithms on state-of-the-art HW/SW platforms.
  - the gained knowledge to explore design trade-offs in real-time performance vs. implementation complexity.
  - the gained knowledge to evaluate the implementation results (e.g. timing, resource usage, power consumption) and correlate them with the corresponding high level design.



Embedded Signal Processing Systems

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

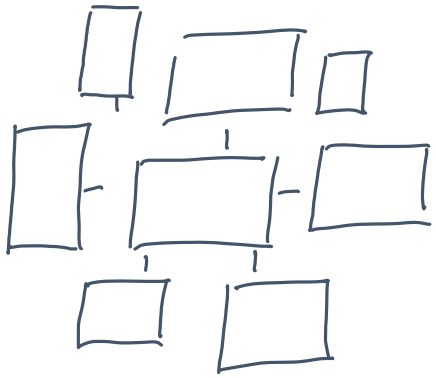
fbeit

Introduction to SystemVerilog

Course Objectives

By the end of this lecture you will be able to

- Understand the basic structure of a DDFS architecture
- Design simple DDFS architecture using SystemVerilog
- Simulate a SystemVerilog based DDFS implementation using Mentor Modelsim



An introductory project ...

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt
h_da
Faculty of Electrical Engineering and Information Technology
fbeit

Problem statement

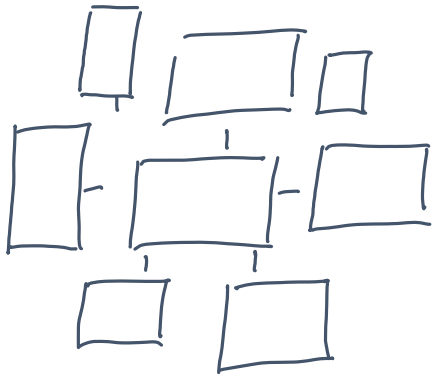
- Let’s assume that we intend to generate a sinusoidal signal? How to implement that task on a general purpose microcontroller?

Problem statement

- Let's assume that we intend to generate a sinusoidal signal? How to implement that task on a general purpose microcontroller?
- What are possible implementation strategies?

Problem statement

- Now assume that we intend to generate a sinusoidal signal using an FPGA based system plattform.
- How to do that?



A simple DDFS architecture

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

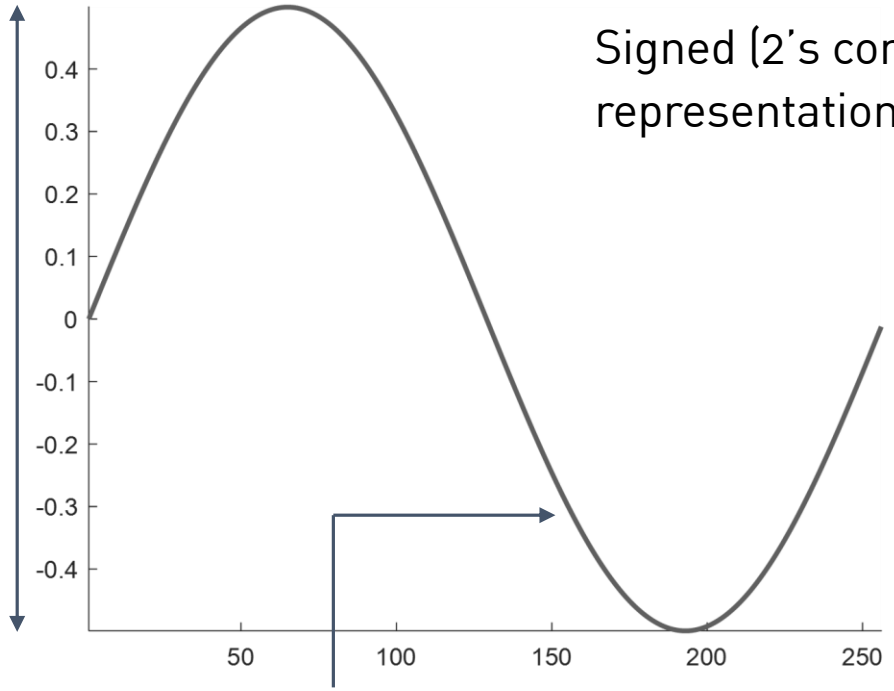
University of Applied Sciences Darmstadt
h_da
Faculty of Electrical Engineering and Information Technology
fbeit

Direct Digital Frequency Synthesis

- How to generate a digital sine wave signal?

Direct Digital Frequency Synthesis

- How to generate a digital sine wave signal?
- Assume $x[n] = A\sin(2\pi f_v n T_S + \phi_v)$



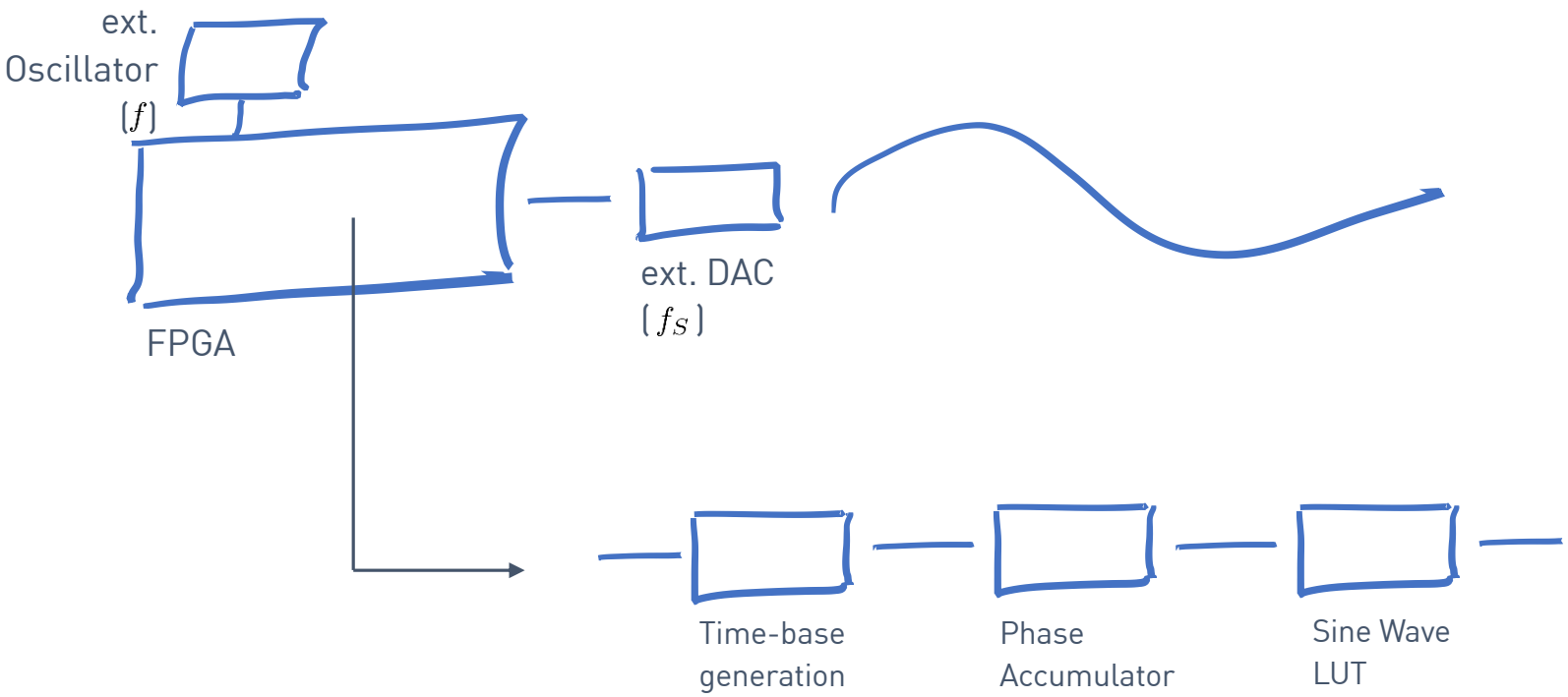
Signed (2's complement), 8-bit representation

- Sampling period

variable frequency f_v
and phase ϕ_v

Direct Digital Frequency Synthesis

- System internals



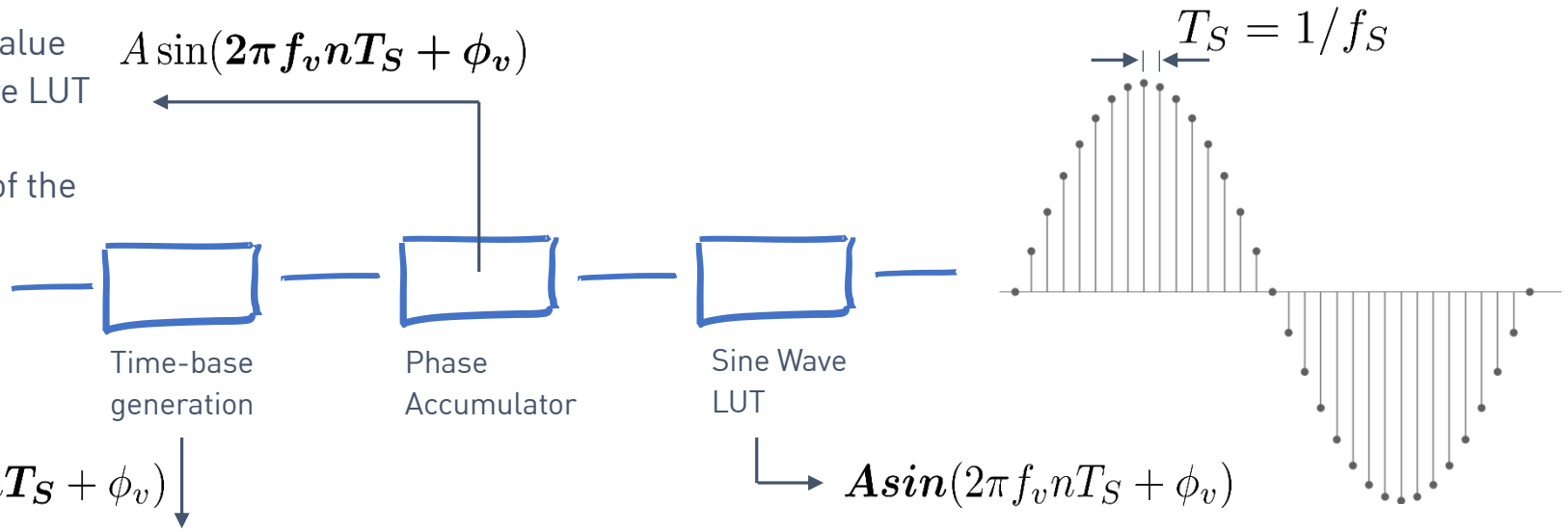
Direct Digital Frequency Synthesis

- System internals – a closer perspective …

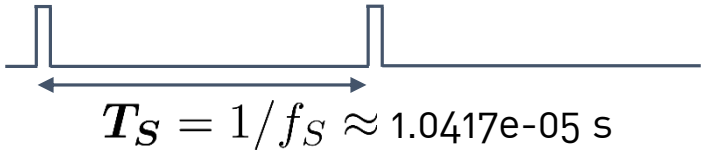
Direct Digital Frequency Synthesis

- System internals – a closer perspective …

- The phase accumulator (PA) finally generates the phase value used to address the sine wave LUT
- The PA determines the **final frequency** and **phase offset** of the output signal



- Driven by the FPGA external 50MHz clock
- Used to activate/enable the phase accumulator at a rate of 96kHz



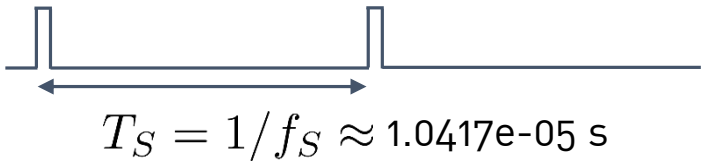
- The sine wave LUT accomplishes the phase to amplitude conversion.
- The sine wave samples are scaled by the amplitude factor A

Direct Digital Frequency Synthesis

- System internals – **Time-base generation**

$$A\sin(2\pi f_v n \boldsymbol{T_S} + \phi_v)$$

- Driven by the FPGA external 50MHz clock
- Used to activate/enable the phase accumulator at a rate of 96kHz



$T_S = 1/f_S \approx$ 1.0417e-05 s

- Generate every 1.0417e-05 s a clock-synchronous (50MHz) pulse.

- This is accomplished using a modulo-**521** counter driven by a 50MHz clock

↳ Modulo-value: $\left[\frac{T_S}{T}\right]_{\text{RTNI}}$: 1.0417e-05 s / 20 ns = 520,833333 = 521

- **RTNI** – round towards the nearest integer
- The bit-width of the Modulo counter is determined by

$$\#\,\text{Bits} = \left[\frac{\log_{10} N}{\log_{10} 2}\right]_{\text{RTNLI}}: 9{,}0251\ \text{Bits} = 10\ \text{Bits}$$

- **RTNLI** – round towards the next largest integer, i.e. always round - up

Direct Digital Frequency Synthesis

```
1. module time_base_gen #(parameter L = 521) (
2.    input logic clk, reset_n,
3.    output logic q
4.    );
5.
6.    localparam BITWIDTH = $clog2(L);
7.
8.    logic [BITWIDTH-1:0] time_base = 0;
9.    always_ff@(posedge clk)
10.       if(reset_n == 1`b0)
11.          time_base <= '0;
12.       else
13.          time_base <= (time_base + 1'b1) % L;
14.
15.   assign q = (time_base == (L - 1'b1)) ? 1 : 0;
16.
17. endmodule
```

- The parameter **L** determines the number of counting cycles
- The counter bit-width is derived using the Verilog $clog2 system function (binary logarithm).
- **Note**: Be aware that we use **L** to implement the modulo counter. However, we use **L-1** to generate the q pulse

$T_S = 1/f_S \approx$ 1.0417e-05 s

[SystemVerilog] Source Code: time_base_gen.sv

Direct Digital Frequency Synthesis

- System internals – **Phase accumulator**

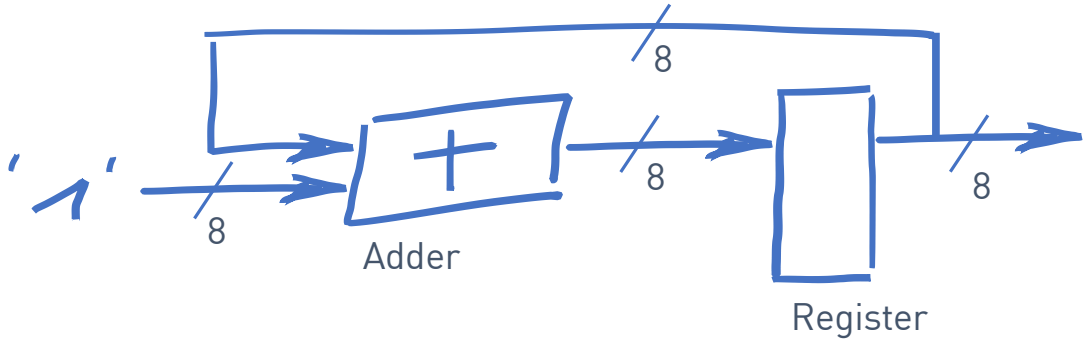
$$A\sin(2\pi f_v n T_S + \phi_v)$$

- The phase accumulator (PA) finally generates the phase value used to address the sine wave LUT.
- The phase value is incremented at a rate of 96kHz.
- The phase resolution (PR) is determined by the number of bits used to represent the phase value:

$$\text{PR} = 360°/2^8$$

- The PA itself determines the final frequency and phase offset of the output signal.
- To simplify the present design, we set the initial phase offset to zero. Moreover, to keep things simple, we increment the PA by $360°/2^8$

- Therefore, the phase accumulator is reduced to a simple 8-bit counter.



- The counter covers the dec. range from zero to 255 and with that the range from 0° to 360°

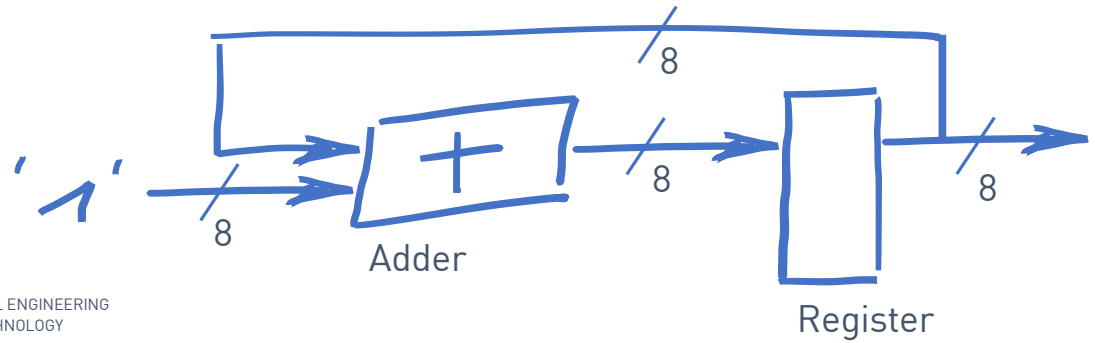
Direct Digital Frequency Synthesis

```
1. module phase_acc (
2.    input logic clk, reset_n, enable
3.    output logic [7:0] q
4.    );
5.
6.    always_ff@(posedge clk)
7.       if(reset_n == 1`b0)
8.          q <= '0;
9.       else if(enable)
10.         q <= q + 1'b1;
11.
12. endmodule
```

[SystemVerilog] Source Code: phase_acc.sv

- Increment the phase value at a rate of 96kHz

$$T_S = 1/f_S \approx 1.0417\text{e-}05 \text{ s}$$

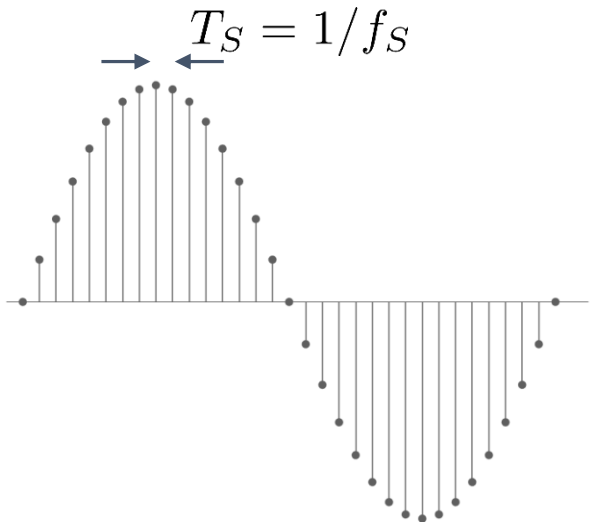


Direct Digital Frequency Synthesis

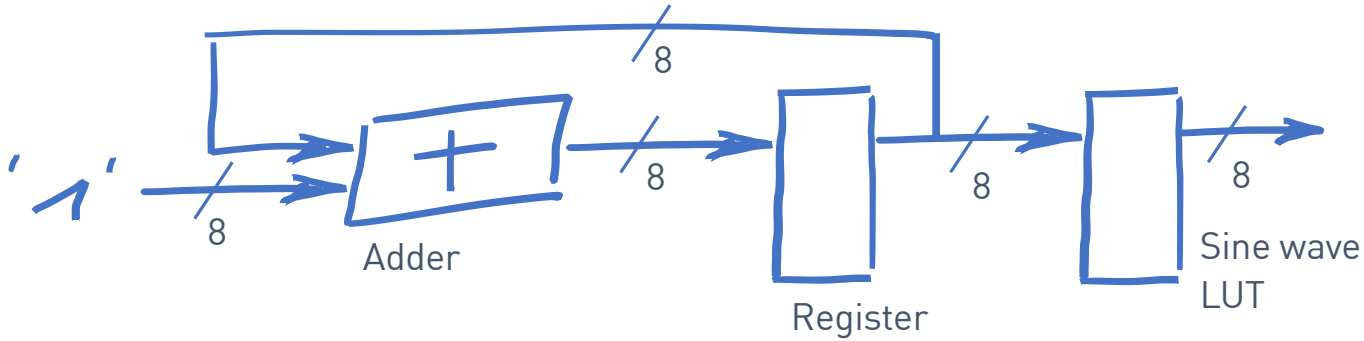
- System internals – **Sine wave LUT**

$$\boldsymbol{A}\boldsymbol{sin}(2\pi f_v n T_S + \phi_v)$$

- The sine wave LUT accomplishes the phase to amplitude conversion



- The Look-Up-Table is just a Read-Only Memory (ROM).
- The LUT is configured as a 8-bit x 256 ROM.
- Thus, the LUT address port has a width of 8-bit and is connected to the 8-bit output of the phase accumulator.



Direct Digital Frequency Synthesis

```
1.  clear all; close all; clc;
2.
3.  n  = linspace(0,1,257);
4.  f  = fi( 0.5*sin(2*pi*1*n),1,8,7);
5.  fq = f(1:256);
6.
7.  figure
8.  set(gcf,'color','w');
9.  stem(fq,'filled','LineWidth',1);
10. box off; axis off;
11.
12. fID = fopen('C:\...\sine_0360_8bit_256.txt','w');
13.
14. for i=1:length(fq)
15.    fprintf(fID,'%4d: q = 8''d%s;\n',i-1,hex(fq(i)));
16. end
```

?

[MATLAB] Source Code: sinewave_lut_init.m

- We use the MATLAB 'fi' function to generate the 8-bit signed format (sign bit, seven fractional bits).
- Why do we declare n with a length of 257 elements despite the fact that we are finally using only 256 sine wave samples?
- The values are formatted and stored in a text-file, so that we can simply copy it to the respective SystemVerilog implementation file. We assume that the LUT output is named 'q':

```
0:   q = 8'h00;
1:   q = 8'h02;
2:   q = 8'h03;
3:   q = 8'h05;
4:   q = 8'h06;
5:   q = 8'h08;
6:   q = 8'h09;
7:   q = 8'h0b;
8:   q = 8'h0c;
9:   q = 8'h0e;
10:  q = 8'h10;
     ...
```

excerpt from: sine_0360_8bit_256.txt

Direct Digital Frequency Synthesis

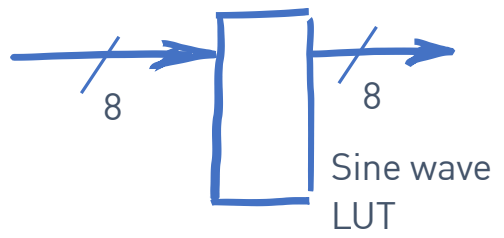
```
1. module sine_wave_lut (
2.    input logic [7:0] address,
3.    output logic [7:0] q
4.    );
5.
6.    always_comb begin
7.       case(address)
8.            0: q = 8'd00;
9.            1: q = 8'd02;
10.           2: q = 8'd03;
11.           3: q = 8'd05;
12.              ...
13.
14.         254: q = 8'dfd;
15.         255: q = 8'dfe;
16.      endcase
17.   end
18. endmodule
```

copy & paste

```
0:    q = 8'h00;
1:    q = 8'h02;
2:    q = 8'h03;
3:    q = 8'h05;
4:    q = 8'h06;
5:    q = 8'h08;
6:    q = 8'h09;
7:    q = 8'h0b;
8:    q = 8'h0c;
9:    q = 8'h0e;
10:   q = 8'h10;
11:   q = 8'h11;
12:   q = 8'h13;
13:   q = 8'h14;
14:   q = 8'h16;
15:   q = 8'h17;
16:   q = 8'h18;
      ...

252:  q = 8'hfa;
253:  q = 8'hfb;
254:  q = 8'hfd;
255:  q = 8'hfe;
```

- There are different ways to implement the sine wave LUT.
- We explore various other approaches in the respective DDFS class.

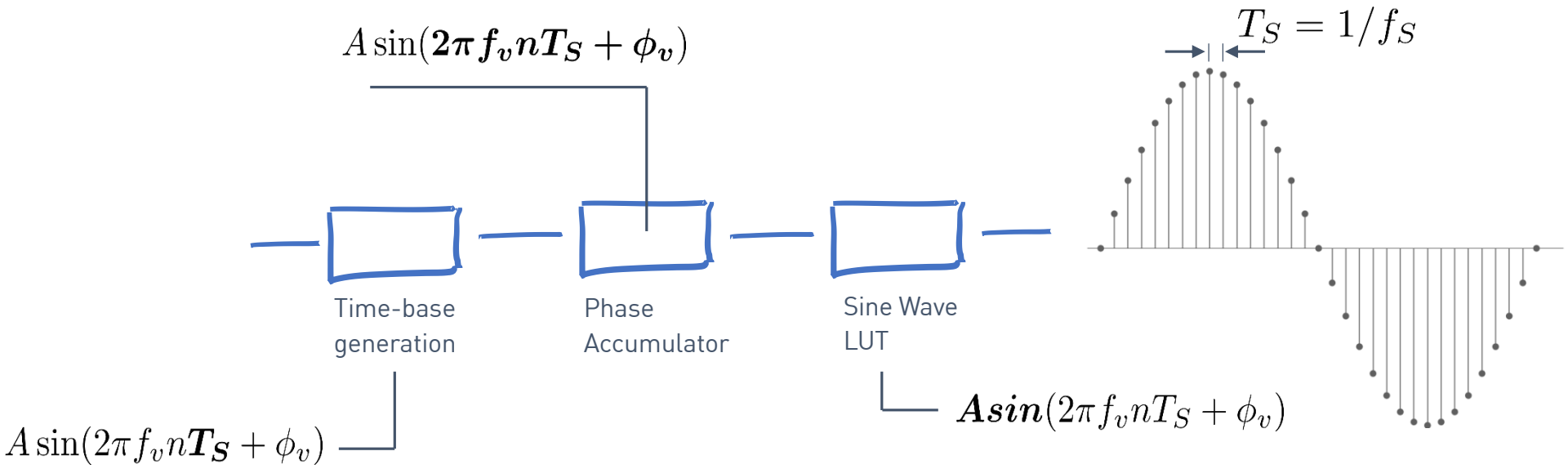


[SystemVerilog] Source Code: sine_wave_lut.sv

excerpt from: sine_0360_8bit_256.txt

Direct Digital Frequency Synthesis

- Finally, we integrate the respective components in a single top-level entity



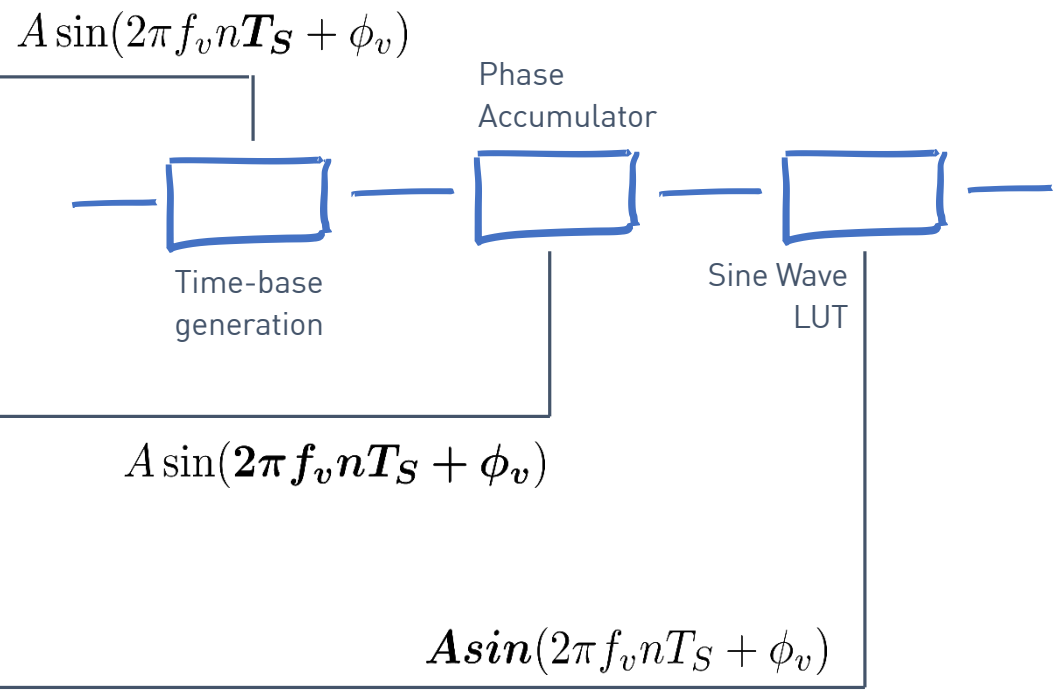
Direct Digital Frequency Synthesis

- At the end of the day …

Direct Digital Frequency Synthesis

```
1. module ddfs (
2.    input logic clk, input logic reset_n,
3.    output logic [7:0] q
4.    );
5.
6.    logic enable;
7.    time_base_gen #(.L(521)) inst_0 (.clk(clk),
8.                                     .reset_n(reset_n),
9.                                     .q(enable));
10.   logic [7:0] lut_address;
11.
12.   phase_acc inst_1 ( .clk(clk),.reset_n(reset_n),
13.                      .enable(enable),
14.                      .q(lut_address));
15.
16.   sine_wave_lut inst_2 (.address(lut_address),
17.                         .q(q));
18. endmodule
```

- We assume that port clk is connected to the 50MHz oscillator



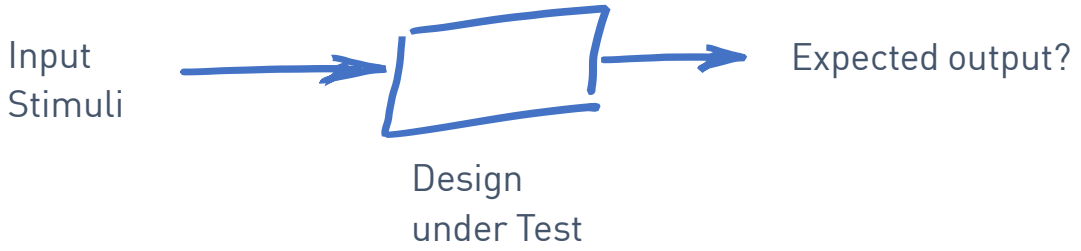
[SystemVerilog] Source Code: sine_wave_lut.sv

Direct Digital Frequency Synthesis

- How to perform a simple verification of the implemented circuitry?

Direct Digital Frequency Synthesis

- Next, we need to design a testbench to simulate and thus verify the DDFS design.
- We use Mentor Modelsim to verify the correct functionality of the circuit. First, we instantiate the DDFS design, supply the circuit with some inputs and then finally observe the respective output behaviour …



Direct Digital Frequency Synthesis

1/2

```
1.  `timescale 1ns/1ps
2.  `define HALF_CLOCK_PERIOD   10
3.  `define RESET_PERIOD       100
4.  `define SIM_DURATION   5000000
5.
6.   module ddfs_tb();
7.
8.      logic [7:0] tb_q;
9.
10.     logic tb_clk = 0;
11.     initial
12.         begin: clock_generation_process
13.             tb_clk = 0;
14.             forever begin
15.                 #`HALF_CLOCK_PERIOD tb_clk = ~tb_clk;
16.             end
17.         end
18.
```

- 50MHz clock stimuli

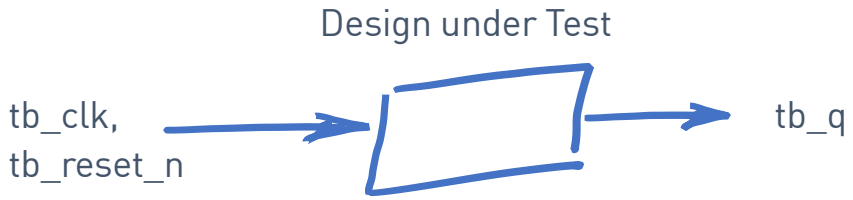
[SystemVerilog] Source Code: ddfs_tb.sv

Direct Digital Frequency Synthesis

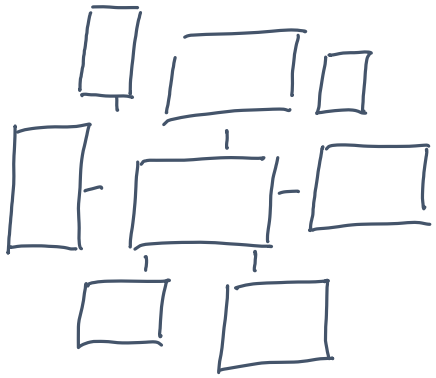
2/2

```
19.     logic tb_reset_n = 0;
20.     initial
21.         begin: testbench_scheduler
22.             $display ("Simulation starts ...");
23.             #`RESET_PERIOD
24.             tb_reset_n = 1;
25.             #`SIM_DURATION
26.             $stop();
27.         end
28.
29.     ddfs inst_dut (
30.             .clk(tb_clk),
31.             .reset_n(tb_reset_n),
32.             .q(tb_q)
33.     );
34.
35. endmodule
```

- A simple testbench scheduler: reset the device and than just wait until it is over ...



[SystemVerilog] Source Code: ddfs_tb.sv



Embedded Signal Processing Systems – Lab@Home

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt
h_da
Faculty of Electrical Engineering and Information Technology
fbeit

Lab@Home

- A quick start guide …

Lab@Home

Using Mentor Modelsim to simulate the DDFS design

- Open a console and create a project directory

```
mkdir ddfs_project
```

- Use your favourite editor to create the previous design. Make sure that all source files are stored in the project directory. For example:

```
gedit time_base_gen.sv &
```

- After creating all files including the testbench, start Modelsim

```
vsim &
```

- Start a new project using

```
File ... New ... project
```

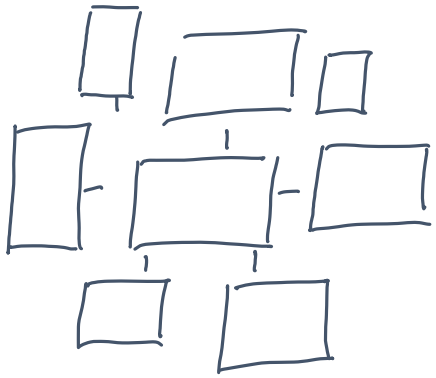
- In the dialog box specify the name and project location and press `OK`
- In the next popup window, Add existing File, and select

```
ddfs_tb.sv
```

Lab@Home

Using Mentor Modelsim to simulate the DDFS design

- Next, we compile the design by
  `Compile using Compile...Compile All`
- You should see a message that states `Compile of ddfs_tb.sv was successful.`
- In the next step, we simulate the design. Choose
  `Simulate ... Start Simulation`
- In the dialog box navigate `to work ... ddfs_tb.sv`
- You should see the testbench signal names in a panel. Select them all and choose
  `Add ... To Wave ... Selected Signals`
- Finally, perform
  `Simulate ... Run ... Run -All.`
- You should see waveforms in the right panel.
- Right-click the name of the `tb_q` and choose `Format...Analog(automatic)`



Lab@Home – Review Questions ...

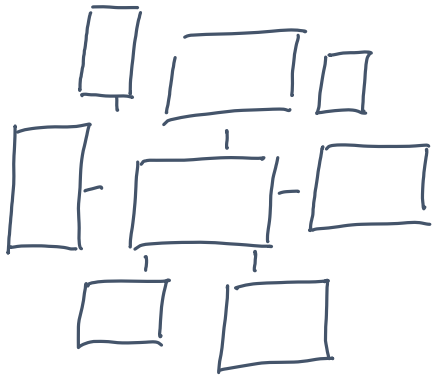
International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt
h_da
Faculty of Electrical Engineering and Information Technology
fbeit

Lab@Home

- Does the simulation duration perfectly fit?
- Discuss possible design changes to reduce the simulation duration while maintain the underlying functionality.
- Calculate the frequency of the generated output signal.
- What design changes are necessary to double the frequency of the output signal?
- Is it necessary to store a full 360° sine wave within the LUT ROM?
- What design changes are necessary to implement a phase offset?



Literature

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob
University of Applied Sciences Darmstadt
h_da
Faculty of Electrical Engineering and Information Technology
fbeit

Introduction to SystemVerilog

Recommended Readings

Books, application notes, ...

- Sutherland, S., "RTL Modeling with SystemVerilog for Simulation and Synthesis: Using SystemVerilog for ASIC and FPGA Design", CreateSpace Independent Publishing Platform, 2017
- Symons, P., "Digital Waveform Generation", Cambridge University Press, 2013.
- Analog Devices, "All About Direct Digital Synthesis", Analog Dialogue 38-08, 2004.